metacognition stuff

Anonymous Author(s)
Affiliation
Address
email

Abstract

TODO: Write abstract last.

1 Introduction

TODO: Write introduction after Section 3 is solid.

2 Background and Related Work

Metacognition is the ability to know one's own knowledge base and reason on it. By virtue of it, a system can tell a lot- whether it can carry out a certain task, whether it can construct the solution of a task from the tasks that it knows, whether it has enough information to provably resolve an issue and many more.

Few attempts have been made for an operational definition of metacognition. Most descriptions are behavioral.

2.1 Calibration in Machine Learning

TODO:

2.2 Metacognition in Large Language Models

TODO: Kadavath et al. 2022 - "Know What You Don't Know" **TODO:** OpenReview 2025 - Functional metacognitive abilities

2.3 The Gap This Work Fills

3 Benchmark Design

3.1 Construct Definition

We attempt to formalize. Following are the routes we could have taken

1. **Second-Order Logic**: It is the closest formalism that directly correlates to these lines. It enables us to make statements on models. So hopefully it has something for us.
2. **MetaProgramming**: Programming languages have consistently tried to achieve this abilities through reflection, templating, etc. Earliest attempts have been that of LISP and 3-LISP. We have failed, but in absolute terms. We still use reflection to useful extents. This can formalize.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

3. **Semantic Understanding of submodules:** This is the most abstract one. This lies at intersection of mechanistic interpretability and Reflection. This will encounter us the Godel Incompleteness that Second Order Logic brings,
4. **Type Theory:** Query levels can tower. Query levels can interject across. The model must be able to handle it. In fact it certainly has to use the lower query levels to be able to describe what is happening in higher query levels. We cannot teach all levels.

3.2 *Reflection Interpretation

Note: Early attempt at formalization. Verify it well.

This interpretation assumes that a model M is trained for a certain task. The model has never been given the information about what it has been trained for. Does the model, reflectively realize, what it can do?

A task $T : \mathcal{D} \to \mathcal{R}$ is defined as a mapping from a domain of inputs to a range of outputs. The model M arrived at through any.

We will consider a binary decision task. The results can be extended to functional tasks. **Check, TCS does it with Functional P algorithms. See why it can do that**

Any decision task can be represented as a first-order logic problem.

We can consider the model to be an ensemble of solutions to tasks.

$$T_b$$

3.3 Task Regimes

3.3.1 Reasoning over abilities

If I train a model for a few tasks, can I start asking questions like

- Given your abilities, which take you the most time to solve. The model cannot fake this by simulating all possible work

- Given your abilities, what more abilities you can train yourself on.

- Given your abilities, where do you find "axiomatic" gaps in. (Toughest genre)

3.3.2 Easier: Reflection

Model pretrained on science, is suddenly introduced to lore of Harry Potter.

- Where do you find inconsistencies in your knowledge base. The model has to reason over its knowledge spaces.

3.3.3 Regime 1: Complete Information

3.3.4 Regime 2: Withheld Information

TODO

3.3.5 Regime 3: Misleading Information

TODO

3.4 Confidence Elicitation

TODO

3.5 Task Generation and Contamination Control

TODO

4 Scoring Methodology

5 Experiments

5.1 Models Evaluated

TODO

5.2 Qualitative Analysis

TODO: Pick the most striking failure case from your runs and show it verbatim.

6 Discussion

TODO

7 Limitations

TODO

8 Conclusion

TODO: Write last.

References